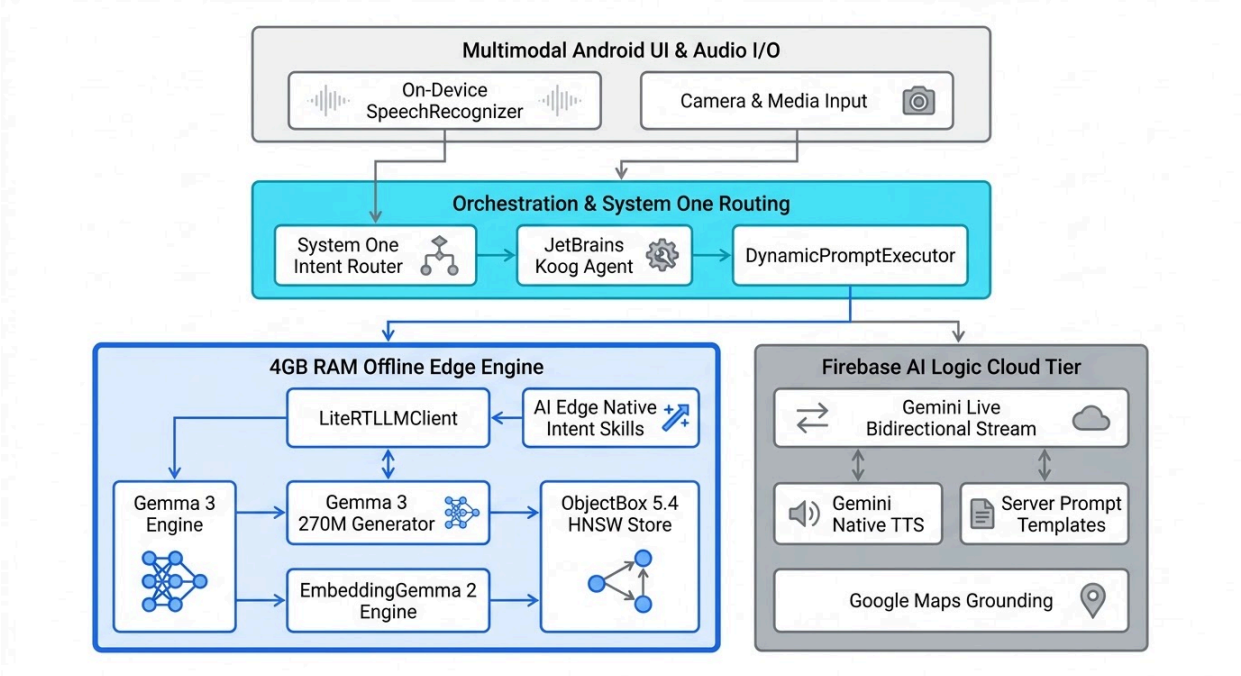
Rearchitecting OpenMMPA for Edge-First Android: A 4GB RAM Reference Architecture with LiteRT-LM, Firebase AI Logic, and JetBrains Koog

Executive Architectural Synthesis: From Cross-Platform Flutter to Native Edge-First Android

Migrating the Open-Multi-Modal-Personal-Assistant (OpenMMPA) from a cross-platform Flutter codebase to a native Kotlin Android application represents a fundamental shift from cloud-dependent middleware to a hardware-accelerated, local-first agentic architecture[1]. In its original implementation, OpenMMPA relied on Dart FFI bindings (objectbox_flutter_libs 4.0.3) for local storage and delegated speech and multimodal processing to external Firebase Cloud Functions (STTSFunctions), which required manual management of audio encodings (TTSEncodings.xlsx) and incurred round-trip network latency on every user interaction[2].

By targeting a native Android environment on a constrained 4GB RAM edge device, the rearchitected system eliminates the STTSFunctions backend entirely while unifying patterns from Google's reference implementations—specifically FriendlyMeals-Android for Firebase AI Logic, JetPacker (android/ai-samples) for multimodal itinerary and voice note workflows, and the Google AI Edge Gallery for local LiteRT-LM execution and Model Context Protocol (MCP) skill orchestration[5].



A critical structural finding across Google's Android AI ecosystem is the deprecation of legacy inference abstractions. As of August 2026, the MediaPipe LLM Inference API (com.google.mediapipe.tasks.genai.llminference) is officially marked @Deprecated and placed in maintenance-only mode[9]. Furthermore, the legacy TensorFlow Lite Interpreter API has been superseded by the LiteRT V2 CompiledModel API (com.google.ai.edge.litert:litert:2.2.0), while generative language and multimodal embedding workloads have migrated to **LiteRT-LM** (com.google.ai.edge.litertlm:litertlm-android)[3]. Consequently, building a modern Android prototype requires bypassing deprecated MediaPipe GenAI wrappers in favor of LiteRT-LM and MLKit GenAI primitives, paired with JetBrains Koog (ai.koog:koog-agents:1.3.0), which includes a dedicated LiteRTLLMClient out of the box[12].

Architectural Layer	Legacy OpenMMPA (Flutter)	Rearchitected Android Edge App (Kotlin)	Primary Reference Repository & Artifact
UI & Application Host	Flutter cross-platform widget tree[1]	Native Android (Jetpack Compose, Android 12+ minSdk 31)[6]	android/ai-samples/jetpacker[6]
Speech & Audio I/O	STTSFunctions Cloud Functions + TTSEncodings.xlsx[4]	Online: Firebase AI Logic Gemini TTS & Gemini Live API; Offline: Android SpeechRecognizer & TextToSpeech[5]	FirebaseExtended/FriendlyMeals-Android[5]
Agent Orchestrator	Custom Dart state management	**JetBrains Koog (1.3.0)** with DynamicPromptExecutor, LiteRTLLMClient, and History Compression[12]	JetBrains/koog (ai.koog:koog-agents:1.3.0)[12]
Decision & Intent Router	Cloud LLM prompt routing	Local **System One Router** (LiteRT V2 CompiledModel BERT Classifier or EmbeddingGemma 2 Cosine Router)[17]	google-ai-edge/litert (litert:2.2.0)[3]

On-Device LLM	None (Cloud-dependent)	**Gemma 3 270M** (.litertlm INT4/INT8) via LiteRT-LM solely for RAG synthesis & local tool calls[10]	google-ai-edge/gallery & LiteRT-LM[8]
Embedding Engine	External / Unbundled[21]	**EmbeddingGemma 2** (270M text-only or 740M modular multimodal) with MRL truncation to 256d[19]	litert-community/embeddinggemma-2-text-270m-litert-lm[19]
Local Vector Database	objectbox_flutter_libs 4.0.3[2]	**ObjectBox Android (5.4.1)** with native Kotlin @HnswIndex[24]	objectbox-kotlin / objectbox-android[2][6]
Tool & Skill Execution	Hardcoded Dart functions	Koog SkillsDiscovery + AI Edge Gallery SKILL.md (run_intent & run_js) + Firebase Function Calling[13]	google-ai-edge/gallery/skills & JetBrains/koog[13]
Security & Governance	Custom API keys / Cloud Functions	**Firebase App Check** (Play Integrity), **Server Prompt Templates**, and **Remote Config**[5]	FirebaseExtended/FriendlyMeals-Android[5]

Hardware Reality Check: The 4GB RAM Budget and LiteRT-LM Runtime Strategy

Operating generative AI models on an edge Android device with 4GB of physical LPDDR4X/LPDDR5 RAM requires rigorous native memory accounting. On Android 12 and higher—the minimum OS baseline required by Google AI Edge Gallery[8]—the Linux kernel, SystemUI, Google Play Services, hardware graphics buffers (Gralloc/ION), and the Android Runtime (ART) zygote consume between 1.8 GB and 2.3 GB of resident memory before the target application launches. Consequently, the Android Low Memory Killer Daemon (lmkd) enforces an effective application ceiling of 1.1 GB to 1.4 GB of combined Java heap and native

malloc/DMA-BUF memory.

This hardware ceiling validates the hypothesis that **Gemini Nano** (backed by Gemma 4 E2B or Gemma 2 2B architectures via AICore and MLKit GenAI) is unsuitable as the primary resident model on a 4GB device. A 2-billion-parameter model quantized to 4-bit weights requires roughly 1.1 GB to 1.3 GB for static weights alone, and expands past 1.6 GB once the KV-cache, activation buffers, and XNNPACK/GPU delegates initialize—triggering immediate out-of-memory (OOM) termination when run alongside a vector database and an embedding model.

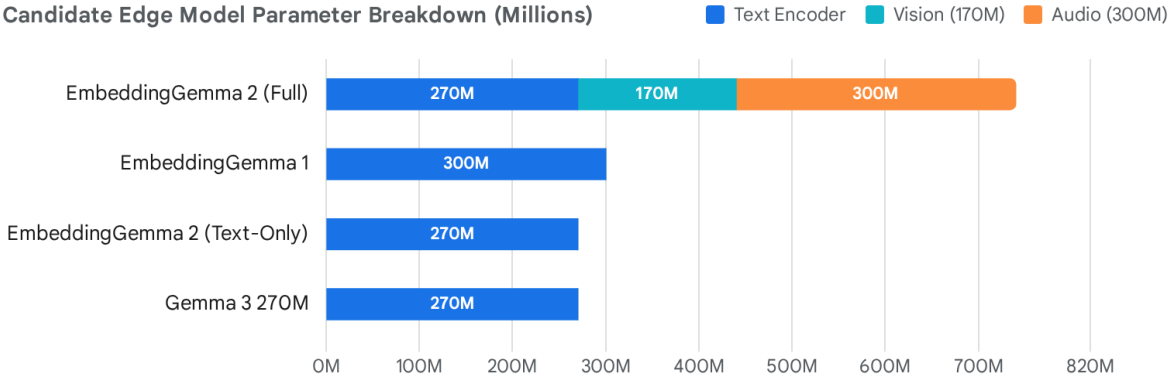
Fitting Generative AI and Multimodal Embeddings Inside a **4GB RAM** Budget

LITERT BENCHMARK · EMBEDDINGGEMMA-300M MIXED PRECISION (E4_A8_F4_P4)

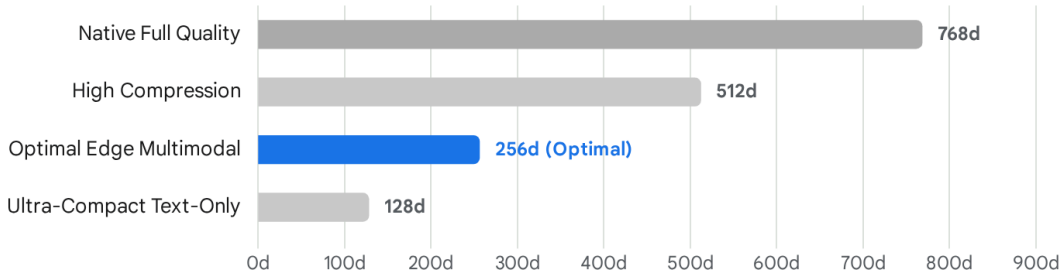
CPU XNNPACK (4 Threads)

Model File Size	Peak RAM Usage	Initialization Latency
196 MB	**333** MB	**2,455** ms
.tflite Flatbuffer Storage	Resident Memory Indicator	One-Time App Launch Cost

Candidate Edge Model Parameter Breakdown (Millions)



EmbeddingGemma 2 Matryoshka Representation Learning (MRL) Dimensions Truncated vector width in ObjectBox



By pairing **Gemma 3 270M** with the modular **270M text encoder** of EmbeddingGemma 2 and truncating vector outputs to **256 dimensions** via Matryoshka Representation Learning, the entire local RAG pipeline remains well under the resident memory ceiling of a 4GB Android device.

Data sources: Google DeepMind (EmbeddingGemma 2), LiteRT Community (300M Benchmark), LiteRT Community (270M Text), LiteRT Community (740M Multimodal), Google Developers Blog (Gemma 3 270M), Ollama Library

Selecting and Configuring the Local Model Pair: Gemma 3 270M and EmbeddingGemma 2

To stay comfortably within a ~650 MB native memory envelope, the architecture pairs **Gemma 3 270M** for text synthesis with **EmbeddingGemma 2** for semantic encoding[19]. Both models

derive from the exact same Gemma 3 foundational architecture and share a unified SentencePiece tokenizer (tokenizer.model), which allows the Android application to load a single tokenizer binary into memory for both embedding and generation tasks[22].
Google DeepMind's **EmbeddingGemma 2** (google/embeddinggemma-2) is an open multimodal embedding model with 740M total parameters, mapping text, code, images, audio, and video into a unified 768-dimensional vector space across an 8K token context window[22]. Crucially, EmbeddingGemma 2 is constructed with a modular architecture comprising three distinct encoders: a 270M-parameter text encoder available standalone as litert-community/embeddinggemma-2-text-270m-litert-lm, a 170M-parameter vision encoder that processes image frames and visual scenes, and a 300M-parameter audio encoder that processes multi-language speech and acoustic events[22]. Because EmbeddingGemma 2 allows developers to toggle off or omit unused encoders at load time[22], the application can dynamically manage its memory footprint on a 4GB device using LiteRT-LM's EmbeddingEngine API[19].

Model Artifact / Hugging Face Checkpoint	**Parameter Breakdown**	**Quantization & Precision Recipe**	**Disk / Flatbuffer Size**	**Peak RAM Footprint**	**Recommended 4GB Android Lifecycle Strategy**
litert-community/embeddinggemma-300m[32]	300M (Text-only v1, T5Gemma init)[32]	Mixed Precision e4_a8_f4_p4 (int4 embed/FFN/proj, int8 attention)[32]	196 MB[32]	333 MB (2,455 ms init on 4-thread XNNPACK CPU)[32]	Legacy baseline; superseded by Embedding Gemma 2 Text-270M[23].
litert-community/embeddinggemma-2-text-270m-litert-lm[23]	270M (Text & Code only)[22]	4-bit / Mixed Precision .litertlm[32]	~175–190 MB[32]	~300–320 MB[32]	**Always-Resident Default:** Co-resides in RAM alongside Gemma 3 270M for instant zero-latenc

					y query & document embedding[19].
litert-community/embeddinggemma-2-740m-litert-lm[37]	740M Total (270M Text + 170M Vision + 300M Audio)[22]	4-bit .litertlm (GPU/NPU backend)[19]	~460–500 MB	~680–750 MB (all encoders active)	**On-Demand / Modular Loading:** Omit audio/vision encoders during normal chat[22], or instantiate Embedding Engine with vision_backend=gpu during background media indexing when Gemma 3 270M is unloaded[19].
Gemma 3 270M (.litertlm)[17]	270M (Instruction-tuned decoder)[20]	4-bit (int4) or Dynamic int8 .litertlm[10]	~180–260 MB	~310–380 MB	**Resident Synthesis Engine:** Loaded via Koog's LiteRTLLMClient for offline RAG synthesis and local tool calling[13].

Critical Engineering Guardrails for EmbeddingGemma 2

Implementing EmbeddingGemma 2 on Android requires strict adherence to three model-specific constraints discovered in its architectural specifications. First, developers must never configure the LiteRT GPU delegate or model weights to execute EmbeddingGemma 2 in float16 precision[22]. EmbeddingGemma 2's internal activation range exceeds the dynamic range of IEEE 754 float16, causing the model to silently emit degraded embeddings or NaN tensors without throwing a runtime exception[22]. When running unquantized subgraphs or activations, bfloat16 (which preserves the 8-bit exponent of float32), float32, or the official 4-bit and mixed-precision (e4_a8_f4_p4) .litertlm builds must be used[22].

Second, because EmbeddingGemma 2 is trained with Matryoshka Representation Learning (MRL) alongside Noise-Contrastive Estimation (NCE), a Global Orthogonal Regularizer (GOR), and Geometric Embedding Distillation[22], its 768-dimensional output vector can be sliced by keeping only the leading $k \in \{512, 256, 128\}$ dimensions and applying L_2 re-normalization[22]. On a 4GB device, truncating vectors to 256 dimensions—or 128 dimensions if the workload is strictly text-only—reduces ObjectBox HNSW graph RAM consumption by 66% to 83% with minimal retrieval quality degradation[22].

Third, EmbeddingGemma 2 relies on asymmetric, task-steered instruction prefixes prepended to text inputs—though never to raw image, audio, or video buffers—to steer representations into the optimal semantic subspace[19]. When indexing documents into ObjectBox, the text must be formatted manually as title: {title} | text: {document}, whereas untitled documents use title: none | text: {document}[22]. When embedding user queries or performing intent classification, task-specific prefixes for search, classification, or clustering must be prepended prior to invoking the EmbeddingEngine[19].

Local Vector Storage: Migrating ObjectBox from Flutter to Native Kotlin

In the original Flutter OpenMMPA repository, local persistence relied on objectbox 4.0.3 and objectbox_flutter_libs 4.0.3[2]. A fundamental architectural clarification is required regarding how ObjectBox handles embeddings: **ObjectBox does not bundle an internal embedding model**[21]. ObjectBox is strictly a high-performance transactional NoSQL object store and Hierarchical Navigable Small World (HNSW) Approximate Nearest Neighbor (ANN) indexing engine; the host application is entirely responsible for transforming raw text, audio, or images into FloatArray vectors via an external model before insertion[21].

Migrating to native Android allows the application to upgrade to **ObjectBox Java/Kotlin/Android 5.4.1**[26], eliminating Dart FFI serialization overhead and enabling direct zero-copy JNI buffer access from Kotlin coroutines. Furthermore, because EmbeddingGemma 2 uses a Global Orthogonal Regularizer (GOR) during training specifically to spread vectors uniformly across the embedding space, its representations are uniquely resilient to quantization and enable highly efficient ANN search in vector databases[31].

In Kotlin, defining the multimodal memory store requires annotating a FloatArray property with @HnswIndex[24]:

```kotlin
package com.openmmpa.android.data.local

import io.objectbox.annotation.Entity
import io.objectbox.annotation.HnswIndex
import io.objectbox.annotation.Id
import io.objectbox.annotation.Index
import io.objectbox.annotation.VectorDistanceType

@Entity
data class MemoryChunkEntity(
    @Id var id: Long = 0,
    @Index var sourceUri: String = "",
    var title: String = "none",
    var contentText: String = "",
    var modalityType: String = "TEXT", // TEXT, IMAGE, AUDIO, VIDEO
    var timestampMillis: Long = System.currentTimeMillis(),

    // Truncated via Matryoshka Representation Learning (MRL) from 768d to 256d
    // Using Cosine distance on L2-normalized EmbeddingGemma 2 vectors
    @HnswIndex(
        dimensions = 256,
        neighborsPerNode = 16, // Reduced from default 30 to conserve RAM on 4GB device
        distanceType = VectorDistanceType.COSINE
    )
    var embedding: FloatArray? = null
)
```

Configuring neighborsPerNode = 16 (the M parameter in HNSW) inside @HnswIndex provides a critical memory optimization[21]. While ObjectBox defaults to neighborsPerNode = 30, lowering M to 16 reduces the in-memory navigational graph connectivity and resource usage on constrained mobile cores while maintaining high recall for personal assistant knowledge bases[21].

Eliminating STTSFunctions via Firebase AI Logic and Local Audio Pipelines

Dropping STTSFunctions removes the maintenance burden of deploying Node.js or Python

Cloud Functions and transcoding audio formats manually[4]. An inspection of FirebaseExtended/FriendlyMeals-Android and the 2026 Firebase AI Logic updates reveals that every capability previously handled by STTSFunctions—plus advanced agentic grounding—is now supported directly within the client-side **Firebase AI Logic SDK**[5].

Specifically, commits to FriendlyMeals-Android (Add Text-To-Speech via Firebase AI Logic #12) demonstrate native Gemini Text-to-Speech (TTS) synthesis directly through the Firebase AI Logic SDK without intermediate Cloud Functions[5]. For interactive multimodal workflows, FriendlyMeals-Android utilizes the liveSession object in Firebase AI Logic to establish a low-latency, bidirectional streaming connection to the **Gemini Live API**[16]. The Android ViewModel captures microphone audio and throttled camera frames—limiting capture to one frame per second converted to a Bitmap on an I/O coroutine dispatcher to conserve bandwidth and memory—and streams them directly to the session while receiving real-time voice responses and FunctionResponsePart client-side tool triggers[16].

Rather than hardcoding verbose system prompts or model identifiers inside the APK, FriendlyMeals-Android stores prompts in the Firebase Console under AI services > AI Logic > Prompt templates and pairs them with **Firebase Remote Config**[5]. This architecture allows developers to iterate on prompts, adjust daily generation caps, or swap Gemini models instantaneously without redeploying the Android client[16]. Furthermore, using TemplateTool within Firebase AI Logic enables spatial intelligence via **Grounding with Google Maps**, biasing requests with the user's current latitude, longitude, day of the week, and time to return structured JSON (StoreFinderResult) localized to the user's immediate surroundings[40]. Because Firebase AI Logic invokes the Gemini Developer API or Vertex AI in Firebase directly from the Android client, pairing SDK initialization with **Firebase App Check** using the **Play Integrity provider** (and DebugAppCheckProvider during emulator or hackathon device testing) is mandatory to attest that traffic originates from an untampered application and prevent billing abuse[5]. When the device transitions offline, the audio pipeline seamlessly falls back to Android's native on-device SpeechRecognizer and system TextToSpeech, keeping the voice interface operational with zero network dependency.

Deep Dive into Architectural Proposals: Offline RAG, System One Routing, and Koog Orchestration

Proposal 1: Constraining Offline Mode to Vector Database Q&A

Restricting the offline operational mode of **Gemma 3 270M** strictly to answering questions over personal content stored in the ObjectBox vector database is not merely a pragmatic compromise—it is an architectural necessity for a 270M-parameter model[20]. While Gemma 3 270M exhibits strong instruction-following capabilities for task-specific workloads relative to its compact size, a 270M parameter weight matrix cannot store reliable open-world parametric knowledge[20]. If prompted with general knowledge queries offline, an ultra-compact model will inevitably confabulate. However, when the system treats Gemma 3 270M strictly as an extractive synthesis reader over the top-K ($K = 3$) chunks retrieved from ObjectBox via EmbeddingGemma 2, the task transforms from open-ended recall into local text compression.

To enforce strict grounding offline, the pipeline should apply a pre-LLM vector distance gate. When ObjectBox executes a nearest-neighbor query via findWithScores(), it returns both the matching MemoryChunkEntity objects and their exact cosine distance scores[21]. If the nearest neighbor distance exceeds a calibrated relevance threshold, the application short-circuits Gemma 3 270M entirely and immediately returns a deterministic fallback response indicating that no matching information was found in local memory while offline. This mechanism conserves battery, prevents hallucination, and eliminates unnecessary LLM decoding latency. Moreover, because EmbeddingGemma 2 natively embeds images, audio recordings, and video sequences into the exact same 256-dimensional MRL vector space as text[19], the offline assistant can also perform zero-LLM multimodal retrieval—matching a natural-language text query directly to an indexed photo or voice note in ObjectBox, mirroring the *Instant Media Search* and *Video Moment Finder* capabilities showcased in the Google AI Edge Gallery[8].

Proposal 2: Implementing a Jev-Style "System One" Decision Model on the Edge

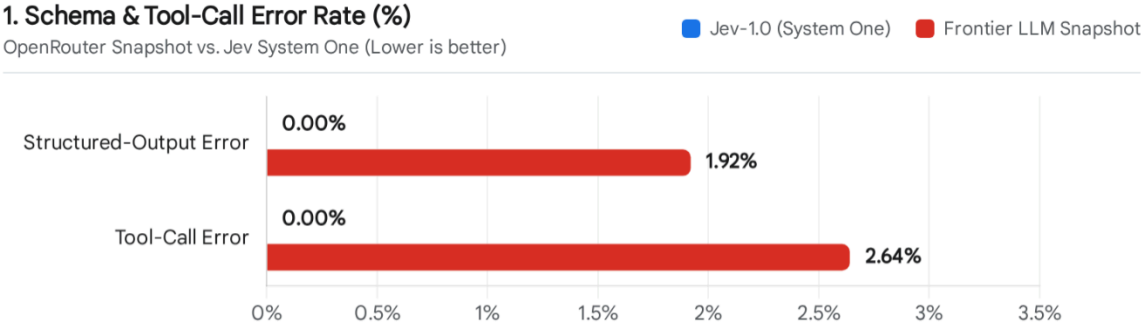
Incorporating a **Jev-like "System One" decision model** addresses the single largest latency and reliability bottleneck in edge agent loops: using a slow, autoregressive generative LLM to make routine routing, classification, and tool-safety decisions[41]. Released by TypeSafe AI in September 2026 (jev-1.13.0), **Jev** is a discriminative transformer model trained on synthetic data via Reinforcement Learning for Calibrated Decisions (RLCD), an objective that penalizes miscalibration likely via Brier score loss terms rather than optimizing for human chat preferences (RLHF)[41]. Instead of generating text token-by-token, Jev takes an unstructured state and evaluates multiple typed questions in parallel in a single pass of 70 to 500 milliseconds, outputting discrete probability distributions and calibrated confidence scores across three multiple-choice primitives: **Choice** (picking from up to 255 categorical options), **Score** (rating an input against ordered levels such as low, medium, and high), and **Noul** (answering a boolean yes-or-no question with a truth probability)[41]. Because its output conforms to the target schema by construction, Jev achieves a 0.00% structured-output and tool-call error rate, compared to 1.92% and 2.64% error rates across standard frontier LLMs[41]. However, **Jev itself is a proprietary, closed-weight model served exclusively through TypeSafe AI's hosted cloud API** at $0.042 per million input tokens, with no published weights or on-device runtime[41]. Consequently, a native LiteRT or MLKit binary of Jev does not exist[41].

System One Decision Routing Outperforms Autoregressive LLM Prompting in Reliability and Speed

Benchmarking Cloud System One & Local Android LiteRT Models against Standard Generative LLMs

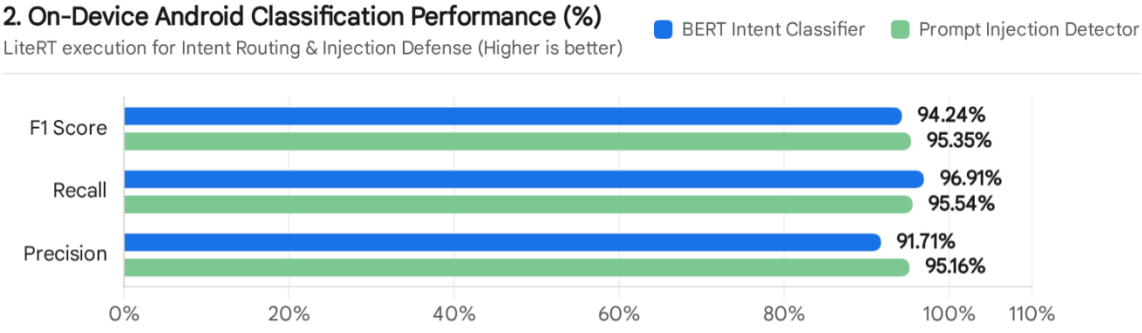
1. Schema & Tool-Call Error Rate (%)

OpenRouter Snapshot vs. Jev System One (Lower is better)



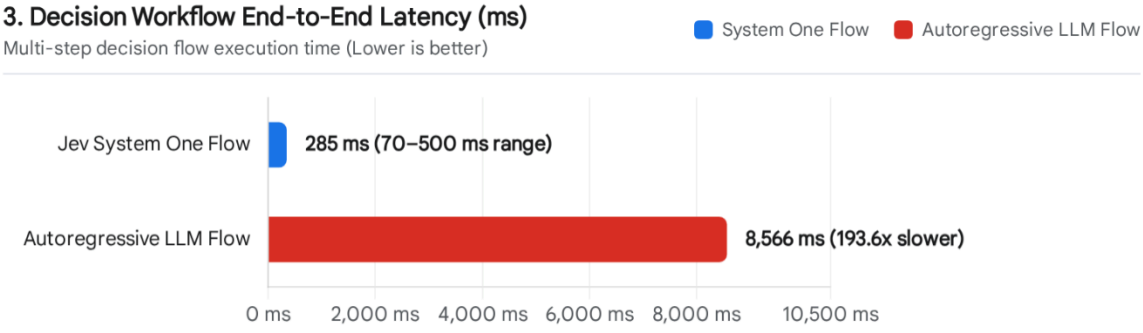
2. On-Device Android Classification Performance (%)

LiteRT execution for Intent Routing & Injection Defense (Higher is better)



3. Decision Workflow End-to-End Latency (ms)

Multi-step decision flow execution time (Lower is better)



While TypeSafe AI's **Jev** eliminates schema errors in the cloud, an on-device **BERT Intent Classifier** running on LiteRT V2 achieves a **0.9424 F1 score** and outperforms few-shot LLM routing with zero network dependency.

Data sources: TypeSafe AI, Wikipedia (Jev AI), arXiv:2510.27140, ProAndroidDev, ACL Anthology

To replicate Jev's **System One** architecture (State $\rightarrow$ [Choice, Score, Noul] with calibrated probabilities) locally on Android via LiteRT, the engineering team has two complementary open-weight implementation paths.

Option A: Zero-Marginal-RAM System One Router via EmbeddingGemma 2

Because **EmbeddingGemma 2 (270M Text)** is already resident in RAM for ObjectBox RAG and natively supports a classification task instruction prefix[19], it can double as a discriminative decision engine with zero additional model weight overhead. At application startup, the system pre-computes and caches 256-dimensional embedding centroids $\{c_1, c_2, \ldots, c_K\}$ for each target route, such as LOCAL_RAG_QUERY, DEVICE_INTENT_ACTION, LIVE_CAMERA_ASSIST, and CLOUD_ESCALATION. When a user utterance x arrives, the router prepends the classification instruction prefix and computes its normalized 256d vector $v(x)$ via LiteRT-LM's EmbeddingEngine[19]. It then evaluates parallel cosine similarities $s_k = v(x)^\top c_k$ and transforms them into a calibrated probability distribution via a temperature-scaled softmax $P(y = k \mid x) = \frac{\exp(s_k/T)}{\sum_{j=1}^{K} \exp(s_j/T)}$, where temperature T is tuned offline using the Brier score to mirror Jev's RLCD calibration[19].

Option B: Dedicated LiteRT V2 CompiledModel BERT Intent Classifier

As independent analyses of Jev observe, its discriminative mechanics closely align with a bidirectional BERT-like encoder architecture[44]. Empirical studies on Android agent pipelines confirm that a **BERT-based Intent Classifier** running locally outperforms LLM few-shot intent classifiers in routing queries, achieving **0.9171 precision, 0.9691 recall, and 0.9424 F1**, while an accompanying prompt-injection safety classifier achieves **0.9516 precision, 0.9554 recall, and 0.9535 F1**[18]. Using the **LiteRT V2 CompiledModel API** (com.google.ai.edge.litert:litert:2.2.0), an 8-bit quantized BERT intent classifier .tflite model can be executed with automatic hardware delegate selection and fallback[3]:

```kotlin
val options = CompiledModel.Options().apply {
    // Attempt NPU acceleration first (Google Tensor / Qualcomm / MediaTek),
    // then MLDrift GPU, falling back automatically to XNNPACK CPU
    setAcceleratorPriority(
        listOf(
            Accelerator.NPU,
            Accelerator.GPU,
            Accelerator.CPU
        )
    )
}
```

```
val systemOneRouter = CompiledModel.create(context, "system_one_bert_q8.tflite", options)
```

If a vendor NPU driver fails to compile a specific subgraph, LiteRT V2 automatically isolates the unsupported nodes and routes them to the optimized XNNPACK CPU backend without crashing the application[17]. Furthermore, by adopting Jev's confidence-gating pattern[41], the Android application inspects the maximum output probability $p_{\max}$ to govern execution: high-confidence decisions ($p_{\max} \geq 0.80$) trigger local execution automatically, mid-confidence decisions ($0.50 \leq p_{\max} < 0.80$) prompt the user for confirmation before executing sensitive Android Intents, and low-confidence decisions ($p_{\max} < 0.50$) escalate the query to Firebase AI Logic when online[41].

Proposal 3: Orchestrating the Hybrid Graph with JetBrains Koog

Selecting **JetBrains Koog** (ai.koog:koog-agents:1.3.0) as the Android orchestration framework is an exceptional architectural fit that eliminates weeks of custom state-machine plumbing[12]. While Koog was designed as a Kotlin Multiplatform agent framework, its 2026 release cycle introduced four capabilities tailored directly to hybrid edge-cloud Android applications[12].

First, Koog introduced a dedicated **LiteRTLLMClient** (#1980) specifically for executing Google's LiteRT models locally on-device, and enhanced it to support **multiple tool calls per response** (#2073)[13]. This allows Gemma 3 270M to plug directly into a Koog AIAgent workflow without custom JNI wrapper code[12]. Second, Koog provides DynamicPromptExecutor and MultiLLMPromptExecutor abstractions that enable an agent to switch LLM providers at any point during a conversation—or reroute dynamically per turn—while adapting and preserving the existing conversation history[12]. Binding both the local LiteRTLLMClient and the cloud GoogleLLMClient / Firebase AI Logic bridge into a single DynamicPromptExecutor allows the System One router to toggle between offline and online execution transparently[12].

Third, on a 4GB RAM device, allowing Gemma 3 270M's KV-cache to expand across long conversations will spike native memory and degrade token generation speeds. Koog incorporates built-in **intelligent history compression** that summarizes and compresses prior conversational turns to optimize token usage while preserving long-running context[12]. Fourth, both google-ai-edge/gallery and JetBrains Koog (#1383, #2160) have converged on the **Agent Skills specification** (SKILL.md)[13]. In google-ai-edge/gallery, skills reside in kebab-case directories containing a SKILL.md file with YAML frontmatter that instructs the local LLM to invoke either run_intent (to trigger built-in Android OS capabilities such as sending messages or setting alarms) or run_js (to run custom JavaScript inside a lightweight hidden WebView via window.ai_edge_gallery_get_result)[28]. Koog's SkillsDiscovery module natively discovers these skill directories, models them as Skill objects, and generates compact skills-catalog prompts in XML, YAML, or JSON for progressive disclosure—ensuring that a tiny model like Gemma 3 270M is only exposed to full tool parameter schemas when a specific skill is activated[13].

Koog 1.3.0 Capability	Technical Mechanism	Benefit on a 4GB RAM Android Device
LiteRTLLMClient (#1980, #2073)[13]	Direct Kotlin binding to Google LiteRT local models with multi-tool parsing[13]	Eliminates custom C++/JNI LLM orchestration code; supports local function calling[13].
DynamicPromptExecutor / MultiLLMPromptExecutor[1][2]	Runtime resolution and hot-swapping between local and cloud LLMClient instances[12]	Routes simple RAG queries to local Gemma 3 270M and complex multimodal tasks to Firebase AI Logic without resetting chat state[12].
Intelligent History Compression[12]	Built-in conversation summarization and token pruning[12]	Caps Gemma 3 270M KV-cache size under 1K tokens, preventing RAM spikes and lmkd kills[12].
Progressive SkillsDiscovery (#2160)[13]	Generates compact XML/YAML/JSON skill catalogs from SKILL.md directories[13]	Prevents prompt bloat on a 270M model by disclosing full tool schemas only upon skill activation[13].
Agent Persistence & Fault Tolerance[12]	Checkpoints graph state across execution nodes with automatic retries[12]	Restores agent workflow seamlessly if Android's OS lifecycle backgrounds or recreates the Activity[12].

When configuring the Android Gradle build for Koog 1.3.0, engineers must target **JDK 17** or higher and explicitly set **Kotlin 2.3.10** or higher in libs.versions.toml, alongside kotlinx-coroutines 1.10.2, kotlinx-serialization 1.10.0, and kotlinx-datetime 0.7.1[12]. Additionally, because Koog 1.0+ decoupled LLMClient constructors from Ktor in favor of KoogHttpClient.Factory, including ai.koog:koog-agents-additions:1.3.0-beta enables auto-discovery of the HTTP transport on Android, while io.github.oshai:kotlin-logging must be declared explicitly if KLogger is referenced[12].

End-to-End Execution Flow and Hackathon Implementation Blueprint

Unified Runtime Execution Flow

At runtime on the 4GB edge device, the rearchitected components operate in a cohesive

four-stage loop. During multimodal ingestion and background indexing—modeled on JetPacker's voice note, itinerary, and expense capture workflows[6]—a background coroutine passes new user notes or media to LiteRT-LM's EmbeddingEngine running embeddinggemma-2-text-270m-litert-lm (or the modular 740m variant when the LLM is idle)[19]. The engine prepends the title: {title} | text: {document} prefix, truncates the output vector to 256 dimensions via MRL, normalizes it in float32 or bfloat16, and persists the MemoryChunkEntity into **ObjectBox 5.4.1**[22].

When the user speaks or types a query, the local **System One Router** (either the LiteRT CompiledModel BERT classifier or the EmbeddingGemma 2 classification head) evaluates three typed primitives in parallel within 30 milliseconds: a Noul safety check (is_safe_input) to screen out prompt injections from untrusted external text, a Choice routing distribution (intent_route) over {RAG_MEMORY_QA, ANDROID_NATIVE_SKILL, LIVE_CLOUD_STREAM, GENERAL_CLOUD_CHAT}, and a Score rating (complexity_level) across {LOW, MEDIUM, HIGH}[17].

If the device is offline—or if the System One router selects RAG_MEMORY_QA with high confidence and low complexity—**JetBrains Koog** dispatches the request to the local LiteRTLLMClient running **Gemma 3 270M**[13]. For memory Q&A, the query is embedded into a 256d vector via EmbeddingGemma 2 and searched against ObjectBox's @HnswIndex[22]. If the top-3 chunks pass the cosine distance gate, Koog appends them to the prompt via UserPromptAugmenter[13] and streams Gemma 3 270M's grounded response to the UI and local TTS. If the router selects ANDROID_NATIVE_SKILL, Koog uses SkillsDiscovery to match the request against local SKILL.md definitions and invokes the run_intent tool to execute the corresponding native Android Intent[13].

Conversely, when the device has network connectivity and the user initiates live camera assistance, requests spatial local search, or triggers a low-confidence score on the local router, Koog's DynamicPromptExecutor routes the turn to **Firebase AI Logic**[5]. Real-time multimodal sessions open a liveSession stream to the **Gemini Live API** with client-side function calling (FunctionResponsePart) capable of reading and writing directly to the local ObjectBox database, while location-sensitive queries invoke Firebase AI Logic **Server Prompt Templates** with **Google Maps Grounding** (TemplateTool) and synthesize spoken replies via **Gemini TTS**[5].

Gradle Dependency Specification (build.gradle.kts)

The following configuration consolidates the exact Maven artifacts required to initialize the Android project:

```Kotlin
// app/build.gradle.kts
android {
    namespace = "com.openmmpa.android"
```

```
    compileSdk = 35

    defaultConfig {
        applicationId = "com.openmmpa.android"
        minSdk = 31 // Android 12+ required by Google AI Edge Gallery & LiteRT NPU
        targetSdk = 35
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17 // Required by JetBrains Koog 1.3.0
        targetCompatibility = JavaVersion.VERSION_17
    }
}

dependencies {
    // 1. JetBrains Koog Agent Orchestrator & LiteRT Client
    implementation("ai.koog:koog-agents:1.3.0")
    implementation("ai.koog:koog-agents-additions:1.3.0-beta")
    implementation("io.github.oshai:kotlin-logging-jvm:7.0.0")

    // 2. Google AI Edge: LiteRT V2 CompiledModel & LiteRT-LM (Replaces deprecated MediaPipe LLM API)
    implementation("com.google.ai.edge.litert:litert:2.2.0")
    implementation("com.google.ai.edge.litert:litert-gpu:2.2.0")
    implementation("com.google.ai.edge.litert:litert-support:2.2.0")
    implementation("com.google.ai.edge.litertlm:litertlm-android:0.1.0")

    // 3. ObjectBox 5.4.1 On-Device HNSW Vector Database
    implementation("io.objectbox:objectbox-kotlin:5.4.1")
    implementation("io.objectbox:objectbox-android:5.4.1")

    // 4. Firebase AI Logic, App Check, & Remote Config (Replaces STTSFunctions)
    implementation(platform("com.google.firebase:firebase-bom:34.3.0"))
    implementation("com.google.firebase:firebase-ai") // Firebase AI Logic (Gemini API, Live, TTS, Grounding)
    implementation("com.google.firebase:firebase-config")
    implementation("com.google.firebase:firebase-appcheck-playintegrity")
    debugImplementation("com.google.firebase:firebase-appcheck-debug")
}
```

Recommended Phased Hackathon Execution Plan

To maximize development velocity during the hackathon and prevent native memory regressions on the 4GB target device, implementation should follow four sequential milestones:

Hackathon Phase	Primary Deliverable	Key Technical Validation & Risk Mitigation
Phase 1: Local Vector & Embedding Spine	ObjectBox 5.4.1 schema (@HnswIndex(dimensions = 256)) + LiteRT-LM EmbeddingEngine with embeddinggemma-2-text-270m-litert-lm[19].	Verify MRL truncation to 256d + L_2 normalization; verify bfloat16/int8 execution (ensuring float16 is disabled so no NaN vectors occur)[22].
Phase 2: System One Router & Offline RAG	Wire **Gemma 3 270M** into Koog's LiteRTLLMClient (1.3.0)[12] with the System One router (EmbeddingGemma 2 classification prefix or LiteRT CompiledModel BERT)[17].	Enforce the ObjectBox cosine distance gate before invoking Gemma 3 270M; enable Koog history compression to keep total app RAM $<$ [12].
Phase 3: Agent Skills & Native Intents	Port 3–4 SKILL.md definitions (run_intent for alarms, notes, calendar) from google-ai-edge/gallery/skills into Koog's SkillsDiscovery module[13].	Validate multi-tool calling in LiteRTLLMClient (#2073) and progressive XML/JSON catalog disclosure[13].
Phase 4: Firebase AI Logic Cloud Augmentation	Integrate FriendlyMeals-Android patterns: Firebase AI Logic **Gemini TTS**, **Gemini Live API** (liveSession), and **Server Prompt Templates** behind Koog's DynamicPromptExecutor[5].	Register the debug secret in **Firebase App Check** (DebugAppCheckProvider)[5] and demonstrate seamless airplane-mode switching between Firebase AI Logic and local ObjectBox RAG.

Works cited

1. RAG: Upgrade text embedding from text-embedding-004 to ... - GitHub, https://github.com/Open-Multi-Modal-Personal-Assistant/OpenMMPA/issues/48
2. [firebase_vertexai]: Multi tool generative AI calls fail with server error,

https://github.com/firebase/flutterfire/issues/16883
3. GitHub - google-ai-edge/LiteRT: LiteRT, successor to TensorFlow, https://github.com/google-ai-edge/litert
4. STTSFunctions/TTSEncodings.xlsx at main - GitHub, https://github.com/Open-Multi-Modal-Personal-Assistant/STTSFunctions/blob/main/TTSEncodings.xlsx
5. FirebaseExtended/FriendlyMeals-Android: The official Firebase, https://github.com/FirebaseExtended/FriendlyMeals-Android
6. ai-samples/jetpacker/README.md at main · android/ai-samples, https://github.com/android/ai-samples/blob/main/jetpacker/README.md
7. gallery/mcp/README.md at main · google-ai-edge/gallery · GitHub, https://github.com/google-ai-edge/gallery/blob/main/mcp/README.md
8. README.md - google-ai-edge/gallery - GitHub, https://github.com/google-ai-edge/gallery/blob/main/README.md
9. LlmInferenceSession.java - mediapipe - GitHub, https://github.com/google-ai-edge/mediapipe/blob/master/mediapipe/tasks/java/com/google/mediapipe/tasks/genai/llminference/LlmInferenceSession.java
10. Point Android readers to LiteRT-LM and mark the MediaPipe LLM, https://huggingface.co/litert-community/Phi-4-mini-instruct/commit/edafb1440931b748aeb7d0ae767109903d3d6d8d
11. LiteRT vs TensorFlow Lite: what changed, plus the old-name new, https://dev.to/john-rocky/litert-vs-tensorflow-lite-what-changed-plus-the-old-name-new-name-cheat-sheet-40nl
12. koog/README.md at develop · JetBrains/koog - GitHub, https://github.com/JetBrains/koog/blob/develop/README.md
13. Releases · JetBrains/koog - GitHub, https://github.com/JetBrains/koog/releases
14. README.md - android/ai-samples - GitHub, https://github.com/android/ai-samples/blob/main/README.md
15. AI Logic - The Firebase Blog, https://firebase.blog/category/ai-logic/
16. Building a cooking assistant with Firebase AI Logic - CodingCat.dev, https://codingcat.dev/post/firebase-2026-05-building-cooking-assistant-ai-logic
17. On-Device AI Series (Part 4): LiteRT | by Oğuzhan Aslan, https://proandroiddev.com/on-device-ai-series-part-4-litert-79a41a3d0c56
18. Proceedings of the 2025 Conference on Empirical Methods in, https://aclanthology.org/volumes/2025.emnlp-industry/
19. Embedding Models | Google AI Edge, https://developers.google.com/edge/litert-lm/embedding_models
20. Introducing Gemma 3 270M: The compact model for hyper-efficient AI, https://developers.googleblog.com/en/introducing-gemma-3-270m/
21. On-Device Vector Search - ObjectBox Docs, https://docs.objectbox.io/on-device-vector-search
22. google/embeddinggemma-2 - Hugging Face, https://huggingface.co/google/embeddinggemma-2
23. litert-community/embeddinggemma-2-text-270m-litert-lm, https://huggingface.co/litert-community/embeddinggemma-2-text-270m-litert-l

m
24. On-device vector database for Android (Java, Kotlin) - ObjectBox, https://objectbox.io/the-on-device-vector-database-for-android-and-java/
25. Stop Calling Cloud APIs: Running Gemini Nano Locally on Android, https://medium.com/@sivaraaj/stop-calling-cloud-apis-running-gemini-nano-locally-on-android-for-zero-latency-ai-e12d3d0633e0
26. ObjectBox Archives, https://objectbox.io/category/mobile-database/objectbox/
27. Java Release History (<= v1.5) - ObjectBox Docs, https://docs.objectbox.io/release-history
28. gallery/skills/README.md at main · google-ai-edge/gallery - GitHub, https://github.com/google-ai-edge/gallery/blob/main/skills/README.md
29. FirebaseExtended/FriendlyMeals-iOS - GitHub, https://github.com/FirebaseExtended/FriendlyMeals-iOS
30. [Support]: Minimal requirements? · Issue #423 · google-ai-edge/gallery, https://github.com/google-ai-edge/gallery/issues/423
31. Gemma explained: EmbeddingGemma Architecture and Recipe, https://developers.googleblog.com/gemma-explained-embeddinggemma-architecture-and-recipe/
32. litert-community/embeddinggemma-300m - Hugging Face, https://huggingface.co/litert-community/embeddinggemma-300m
33. kamalkraj/embeddinggemma-300m-litert - Hugging Face, https://huggingface.co/kamalkraj/embeddinggemma-300m-litert
34. google/embeddinggemma-2 · Hugging Face : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1wz5va3/googleembeddinggemma2_hugging_face/
35. Paper page - EmbeddingGemma 2: Unified and Efficient Multimodal, https://huggingface.co/papers/2610.37364
36. embeddinggemma - Ollama, https://ollama.com/library/embeddinggemma
37. litert-community/embeddinggemma-2-740m-litert-lm - Hugging Face, https://huggingface.co/litert-community/embeddinggemma-2-740m-litert-lm
38. Gemma 3 270M: A Guide With Demo Project (Android App), https://www.datacamp.com/tutorial/gemma-3-270m
39. Android Archives - ObjectBox, https://objectbox.io/category/android/
40. Bringing Google Maps to Friendly Meals with Firebase AI Logic, https://dev.to/codingcatdev/bringing-google-maps-to-friendly-meals-with-firebase-ai-logic-web-architecture-implementation-4j58
41. TypeSafe AI's Jev: What "System One Models" Actually Are, https://www.truefoundry.com/blog/typesafe-ai-jev
42. What Is Jev? A Guide to TypeSafe AI's System One Model - LangChain, https://www.langchain.com/blog/building-a-harness-with-jev
43. Jev by TypeSafe AI. Machine Native Intelligence | by Cobus Greyling, https://cobusgreyling.medium.com/jev-by-typesafe-ai-4846aaca3186
44. Jev (AI model) - Wikipedia, https://en.wikipedia.org/wiki/Jev_(AI_model)
45. TypeSafe AI: Home, https://typesafe.ai/
46. Exploiting Mobile LLM Agents as Confused Deputies via Non-App,

https://arxiv.org/html/2510.27140v3
47. Koog is a JVM (Java and Kotlin) framework for building ... - GitHub, https://github.com/JetBrains/koog